

One rule base, three entity types

formal guardrails for LLM agents, act V: rules across entities

Relay's cases now carry ruled threads and evidence: a comment's legality depends on its case's live state and org — decided in the kernel, which joins the parent. The client never computes context.

With pre-registered predictions, one un-predicted fail-open lesson, screenshots of internal notes that do not exist for customers, and forged requests refused by name. August 2026.

No formal-methods background assumed — the four concepts you need are on slide 3.

Agents now out-write our ability to check

- LLM agents produce code faster than humans can meaningfully review it. Human attention is the bottleneck — and it samples, it does not cover.
- Tests also sample: they check the runs you thought of. The bugs that matter live in the interactions you did not think of.
- If we want agents (and background jobs, and time) to act autonomously, we need a way to say what must never break — and have a machine enforce it.

The proposal

The developer writes down intent as rules and invariants. Agents write the rest. Reasoning engines sit between them and arbitrate: every change must provably keep the rules.

Code becomes cheap to regenerate. The spec becomes the durable asset.

Setting: a regular web SaaS application — CRUD/API handlers, authorization, tenancy, workflows, background jobs. Not kernels, crypto, or avionics: every technique here is judged against ordinary product teams, CI budgets, and mainstream languages at the edges.

All the formal methods you need today

Invariant

A sentence about the system that must be true at every moment. Example: “a customer never sees another organization's cases.” You already write these — in comments, runbooks and post-mortems. Here they become machine-checkable.

Model

A small, faithful board-game version of your system: its states and legal moves. Here the “model” is not beside the system — the rule base IS both the model and the running program.

Checker

A tireless adversary (here: the Z3 solver). It considers every situation the rules allow — tens of thousands you would never test — hunting for one that breaks an invariant.

Counterexample

The checker's proof of failure: the exact situation that breaks the rule, with the rule that allowed it named. Not “something is wrong” but “an anonymous visitor assigned to a task can start it, via assignee_moves.”

Proofs beside the code: it works — and it has a tax

2 protocols falsified, then proven

P1: the model checker beat a careful engineer twice on an online DB migration. Version 3 survived — and was later proven inductively, for any system size, and live (tracks I, J, M).

59 anomalies per run

P3: skip the guard against real Postgres under concurrent load and the model's predicted anomaly appears 59×/run; 0 with the proven choreography.

1 round to the full fix

P4: same bug, same model, same generic prompt — a stronger frozen gate turned a partial fix into the full fix. Gate strength beats prompt steering.

3.94× faster, never wrong

P5: an agent optimized the CMS freely inside the frozen gate; two correctness-breaking “optimizations” were absorbed mechanically.

Proofs at five levels

Inductive invariants, parameterized (UPDR), liveness under fairness, noninterference via self-composition, a Dafny proof of real session code (tracks I–M).

The residual tax

Every result above polices the model↔code boundary: MBT, trace validation, extraction fidelity. Act II removed that boundary for the ruled part of the system.

Language scouting (notes 11–12): of five ways proofs can meet code, “verified engine + small analyzable DSL” — Cedar's shape — ranked first. Everything after act I builds exactly that.

The rule base is the program; the solver is the reviewer

- One artifact: rules.yaml — roles, lifecycle, allow/deny rules with their ticket sentences. The engine serving it is domain-free; refusals are 403s that NAME the rule.
- One condition grammar, two backends: the analyzer's Z3 view and the server's runtime view are the same parsed rules — agreement checked exhaustively, so spec↔code drift is closed by construction.
- The gate is frozen for agents: safety (EVERY situation) + possibility (SOME — “the safest system does nothing” fails) + feature runs with refusals expected by name + gated lifecycle entries.

```
$ analyze rulesets/cms-buggy --gate rulesets/cms # sabotage
FAIL DEAD allow rule 'editor_read_all' - silently masked
FAIL S2_separation_of_duties counterexample + granting rule
FAIL P2_review_by_non_author: IMPOSSIBLE - strict_privacy
VERDICT: FAIL (7 findings) # 4 detector kinds fire at once
```

Evidence so far (notes 13–14)

- Sabotaged CMS: 7 named findings; naive import extension: 5.
- Background jobs are just actors — a compromised importer is contained by policy, provably.
- Domain transfer: receivables cost the engine ~100 generic lines (time); Flowdeck ~60 (relations).
- Redundant rules proven dead and deleted: the solver shrinks rule bases, reversing 1980s rule rot.

Act III built a full product (Flowdeck) to measure the developer experience — and surfaced the UI tension. Act IV resolved it with the kernel boundary. Act V, this deck's lead, is what came next.

Flowdeck: the loop is fast and honest — the UI was the tension

1

typo = named load error; the author's dead-rule prediction FALSIFIED by the solver.

2

TWO REAL AUTHORIZATION HOLES (right) + two probes naming the wrong deny.

3

containment deny proven DEAD after the fix — deleted; S11 proves it universally.

4

PASS (0 findings). Tickets → green gate ≈ 1 hour, none of it debugging.

34,560 situations

0.18 s / round

2 real holes

round-2 draft gated forever

```
$ analyze rulesets/taskboard      # round 2, 0.18 s
FAIL S2_no_anonymous_access
  counterexample: {role: anonymous, action: start,
    assigned_to_me: true, ...}
  allowed by: assignee_moves
FAIL S5_assignee_works
  counterexample: {role: admin, action: submit,
    assigned_to_me: false, ...}
  allowed by: admin_oversees
```

Interaction bugs, not typos: each rule read fine alone. Nobody writes the test where a task is assigned to “anonymous”; the $\forall$ -check considers all 34,560 situations and names the granting rule. Full journal: [examples/taskboard/DEVLOG.md](#).

Flowdeck's UI was DERIVED from the rule base — zero app code, boards and buttons for free. Great demo, wrong product surface: a real product's UX is not a reflection of its policy vocabulary. The developer redirected.

The boundary moved into a kernel API; the UI went free

FREE — app.py (hand-written htmx, ~760 lines)

Queues, badges, toasts, forms, the thread. Agent-authored, restyled at will. Zero policy lives here — so no review of it can ever be about policy. Its product UX (a cross-state “SLA breached” queue, assign-to-me, locked-action disclosures) is nothing a reflected UI could invent.

BOUNDARY — engine/kernel.py (+ the lint that holds it)

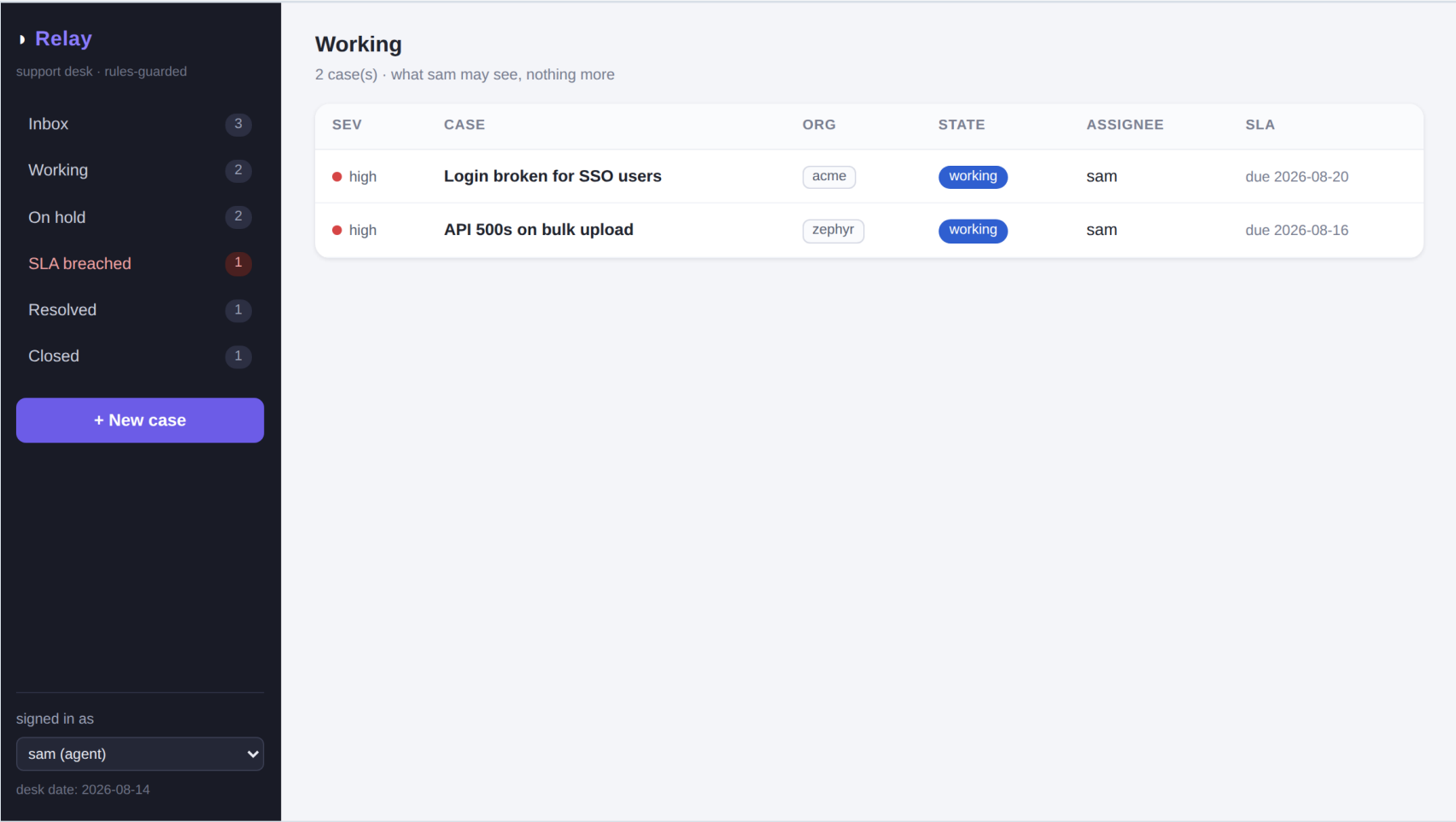
visible / get / create / act / edit / delete — every call decided by the rule base BEFORE the store is touched; refusals are typed Denied values naming the rule; decide()/affordances() are pure queries for rendering. Edits are decided twice (current row AND the row it would become — the tenant-escape edit died here). App code imports engine.kernel and NOTHING beneath it: analysis.boundary fails CI otherwise, and the preserved bypass_variant/ must keep FAILING it.

ANALYZED — rulesets/<app>/ (rules + frozen gate)

The program: allow/deny rules from tickets. Z3 reviews every change: dead rules, $\forall$ -safety, $\exists$ -possibility, lifecycle, frozen features with refusals by name. Relay round 1: PASS with pre-registered predictions held; engine domain growth for the 4th domain: 0 lines.

The tests forge every request the UI hides — the robot triaging, a non-assignee resolving, a customer re-tenanting her case — and all bounce off the kernel, 403 by rule name, nothing written. Hiding a button changes nothing; that is the whole point of the boundary.

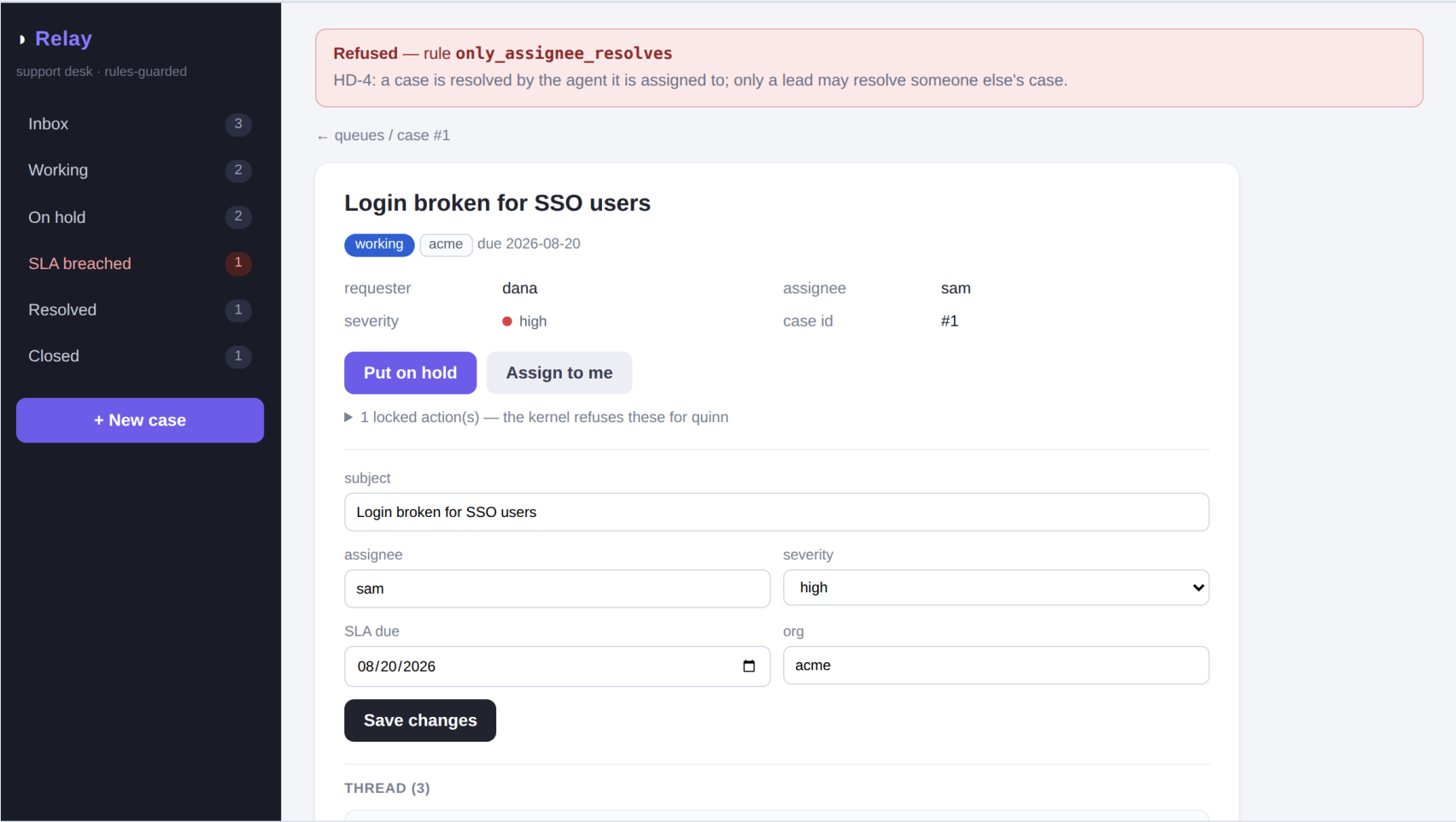
Nobody generated this — and nobody reviewed it for policy



The agent-written surface: dark sidebar, queue navigation with live per-persona counts (the list IS the read rule), severity dots, org chips, SLA badges. htmx partial swaps against the kernel — ~760 lines of UI in which no policy can live.

python examples/helpdesk/app.py → http://127.0.0.1:8810/ — the demo desk is seeded THROUGH the kernel as the real personas, so a seed that violates policy cannot exist (the breached seed case had to route its resolution through the lead).

Press the locked button anyway: the ticket sentence answers



quinn (staff, not the assignee) presses the locked Resolve: 403 — Refused: rule only_assignee_resolves, “HD-4: a case is resolved by the agent it is assigned to; only a lead may resolve someone else's case.” Ticket → rule id → solver vocabulary → typed Denied → toast: one unbroken chain of names.

Affordances, not permissions: one kernel call powers the allowed buttons AND the locked disclosures; the presentation philosophy is the UI author's choice, the decision never is.

“Comments and attachments are context-sensitive” — the wall comes down

The previous deck's own sharp-edge slide:

“One entity per rule base — Relay's cases fit one rule base; case comments/attachments would force relations between ruled entities — N rule bases + client joins, or engine work.”

The developer picked: engine work. “Rules covering multiple entity types is a must.”

Why client joins were never an option

Whoever computes “may dana post here?” from two rule bases plus a join IS an enforcement point — and guardrail 10 exists to keep the UI from ever being one. If relations live in the client, the boundary is gone.

What “context-sensitive” means concretely: a comment's legality depends on the CASE it belongs to — its live state and its org. “No comments on a closed case”, “the org wall extends to the thread”, “internal notes are staff-only” are rules about a relation between two entity types:

```
- id: sealed_thread
  entity: [comment, attachment]
  effect: deny
  when: 'parent.state == "closed"
        and action != "read"'
```

```
- id: org_walls_thread
  entity: [comment, attachment]
  effect: deny
  when: 'actor.role == "customer"
        and not parent.same_org'
```

parent.state and parent.same_org are the child's window onto its parent — the whole relation mechanism.

Children declare a window onto their parent; the kernel does the join

```
# rules.yaml — ONE rule base, three entity types
entity: case           # the root, unchanged
children:
  - entity: comment
    states: [posted, redacted]
    fields: [body, internal]
    context: [state, same_org] # opt-in, per atom
    lifecycle:
      transitions:
        - {action: post, from: none, to: posted}
        - {action: redact, from: posted, to: redacted}
  - entity: attachment
    states: [attached, removed]
    fields: [filename]
    context: [state, same_org]
```

The mechanism, in four facts

- Each entity gets its OWN situation vocabulary; `context:` imports parent atoms one by one (every boolean doubles the space — the budget discipline survives): parent.state, parent.is_author, any parent projection.
- Rules and gate properties carry an entity: tag, defaulting to the root — growing children touched ZERO existing rules and zero frozen gate lines.
- The kernel joins the live parent row into every child decision: create(actor, fields, entity="comment", parent_id=case). The client cannot compute context wrong because it never computes it.
- Deliberate limits, stated up front: one level of nesting, one parent per child, NO aggregates ("close only if no open comments" is not expressible — see the sharp edges).

Single-entity rule bases are the unchanged degenerate case: cms, tickets, receivables and Flowdeck replayed green on the rewritten engine BEFORE anything new was written. Back-compat proven, not assumed.

Close the case; the very next post meets the deny — live

```
# the same call, before and after the case closes
>>> desk.create(priya, {"body": "confirmed fixed"},
...      entity="comment", parent_id=case_id)
{'id': 7, 'state': 'posted', ...}    # resolved: talks

>>> desk.act(noor, "close", case_id) # the QA step
>>> desk.create(priya, {"body": "one more thing"},
...      entity="comment", parent_id=case_id)
kernel.Denied: denied by sealed_thread

>>> desk.visible(priya, entity="comment",
...      parent_id=case_id)
[<the whole thread>]    # readable forever (P13)
```

Why this is the interesting part

- One deny, two entities, four verbs sealed: posting, redaction, attaching, removal — and reads deliberately NOT.
- The industry alternative is a `sealed` flag denormalized onto every child row, plus the sync bugs it breeds. Here there is nothing to sync: child rules read the LIVE parent.
- A resolved case still talks (the customer may dispute, HD-4) but takes no new evidence (fresh_evidence_only) — two different context rules, one vocabulary.
- All of it frozen: feat_sealed_evidence replays 19 steps through resolve → close, refusals expected by name.

The seed data lives under the same law: the demo threads are written BEFORE their case closes, because sealed_thread refuses late seed comments exactly as it refuses late real ones. The same seal meets forged HTTP on slide 18: POST into the closed case → 403, sealed_thread, nothing written.

Round 2: predictions registered first — then PASS

```
$ analyze rulesets/helpdesk          # round 2, ~0.4 s
```

```
rules: 33 (16 deny, 17 allow) | entities: 3
situation space: 37200 (case: 19200,
  comment: 12000, attachment: 6000)
```

```
ok: all 33 rules are effectual
```

```
ok [comment]    S14_threads_follow_org_walls ...
```

```
ok [comment]    S17_comments_immutable (no deny rule)
```

```
ok [attachment] S22_evidence_only_while_working ...
```

```
ok [comment] P13_thread_outlives_closure
```

```
witness: {action: read, parent.state: closed, ...}
```

```
ok: 11 transitions live, all 9 states reachable,
  gated entries respected
```

```
ok feat_thread (14) feat_sealed_evidence (19) ...
```

```
VERDICT: PASS (0 findings)
```

Pre-registered in the DEVLOG — all held

- P-c: zero dead rules again. The new containments (comments never edited, only leads redact, the robot never reads back, removal is author-or-lead) again have NO deny rules — S17/S18/S19/S23/S24/S26 prove each universally.
- P-d: the 13 case $\forall$ -properties and 9 case witnesses are byte-identical and still green.
- P-e: engine domain lines for the thread + evidence: 0 — nothing in the engine says comment or seal.
- Unpredicted gain: P13's witness — the solver picked a CLOSED parent to prove the record survives. “Readable after the parent moved on” used to be unstatable; parent.state made it a one-liner.

Every check runs per entity, in each entity's own situation space — a child's includes every parent-context combination a rule could ever see. Two rounds, two green gates, five predictions held — and the one lesson nobody predicted is the next slide.

Untagged global denies fail OPEN — caught, tagged, pinned

```
# the ONE existing rule this episode touched:
- id: deny_inactive
  description: "A deactivated account can
    do nothing at all."
  entity: [case, comment, attachment]  # ← new
  effect: deny
  when: 'not actor.active'

# pinned the same hour, per child:
- id: S28_inactive_locked_out_of_threads
  entity: comment
  requires: 'actor.active'
```

```
$ # verification: untag the deny, re-run the gate
FAIL [comment] S28_inactive_locked_out_of_threads
  counterexample: {role: customer, action: post,
    active: false, parent.same_org: true, ...}
FAIL [attachment] S29_inactive_locked_out_of_evidence
FAIL feat_thread: step 6 (lex read):
  expected deny by deny_inactive, got allow
VERDICT: FAIL (3 findings)
```

- Untagged rules apply to the ROOT only. That default fails SAFE for allows (a child grants nothing by silence) — and fails OPEN for global denies: without the tag, a deactivated customer could have posted comments.
- The fix is one line; the lesson is mechanical: S28/S29 joined the gate, and the negative direction was verified by temporarily untagging the rule — the analyzer produced exactly the counterexamples the bug would have shipped.
- Recorded as guardrail 11(a) and falsifier ME-5: an actor-only deny not tagged for every entity deserves a load-time warning. This slide exists because guardrail 7 says falsified assumptions get recorded, not smoothed over.

sam sees THREAD (3): the internal note, amber and labeled

SLA breached1

Resolved1

Closed1

+ New case

Put on hold

Resolve

subject

Login broken for SSO users

assignee

sam

severity

high

SLA due

08/20/2026

org

acme

Save changes

THREAD (3)

dana

Affects every SSO user since the 09:00 deploy — password logins still fine.

sam internal

Suspect SAML clock skew after last night's cert rotation.

postbot

Fwd: user report — login loop on the EU tenant.

Write to the thread...

Post

☐ internal note (never reaches the customer)

The thread renders kernel lists: dana's report, sam's internal note ("Suspect SAML clock skew..."), the robot's filed email. The internal checkbox next to Post is shown to staff as a presentation choice — the kernel, not the checkbox, is what keeps customers out.

Every comment card, tag and button comes from `kernel.visible/affordances(entity="comment")` — the UI invented the layout, the rules decided the content.

dana sees THREAD (2): the internal note does not exist

Working1

On hold1

SLA breached1

Resolved1

Closed1

+ New case

workingacmedue 2026-08-20

requesterdana

assigneesam

severityhigh

case id#1

2 locked action(s) — the kernel refuses these for dana

subject

Login broken for SSO users

assigneesam

severityhigh

SLA due08/20/2026

orgacme

Save changes

THREAD (2)

dana

Affects every SSO user since the 09:00 deploy — password logins still fine.

postbot

Fwd: user report — login loop on the EU tenant.

Write to the thread...

Post

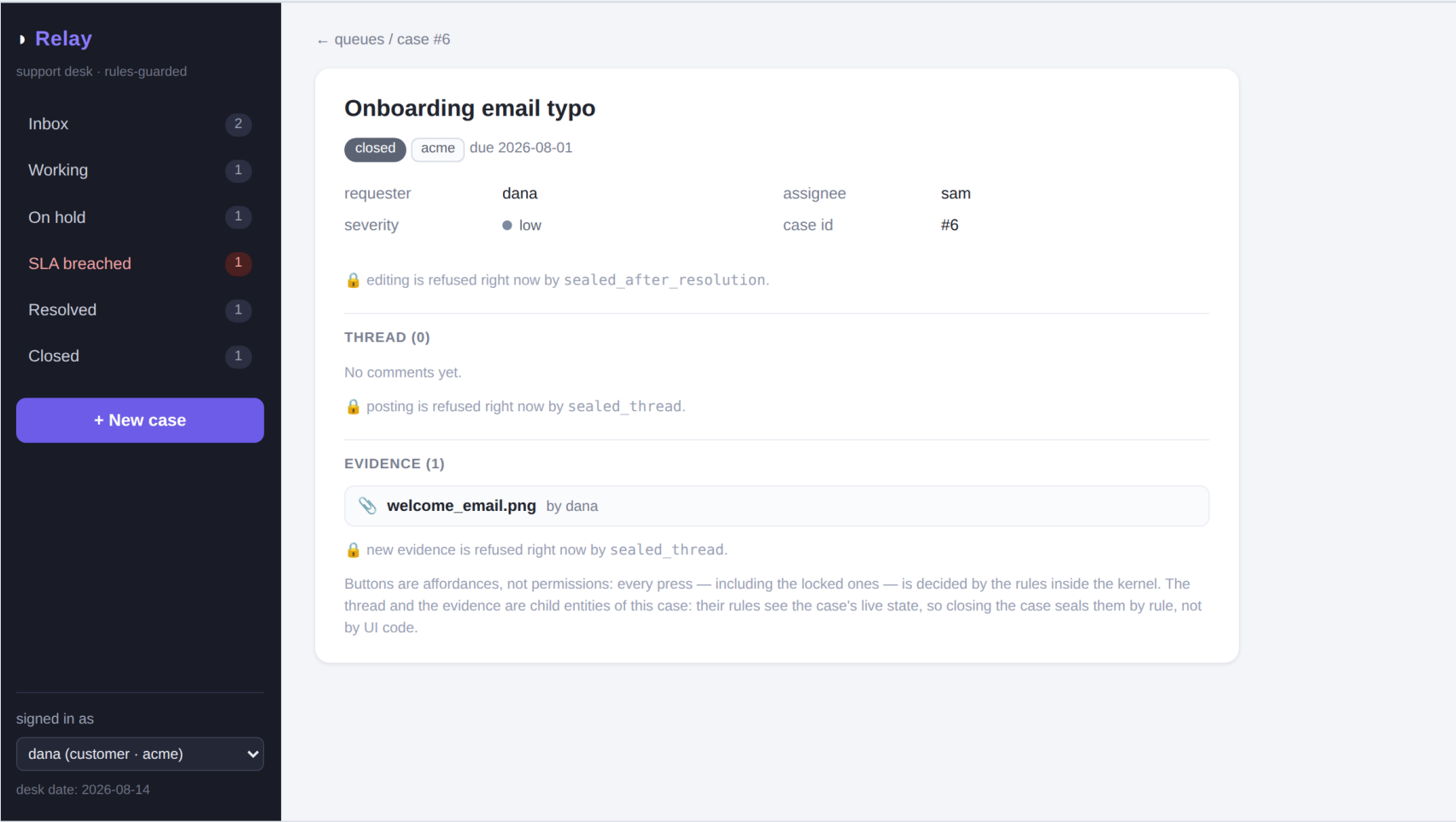
Same case, same thread — one comment fewer, no internal toggle. Not CSS hiding: `visible()` never returned the note, `get()` by forged id raises `Denied(internal_is_staff_only)`, and the forged POST with `internal=yes` comes back 403. S15 proves no allowed action ever touches an internal note outside staff.

The count in the heading is the read rule made visible: THREAD (3) vs THREAD (2) is the whole architecture in two characters.

Formal guardrails for LLM agents · multi-entity rules · 2026-08-14

16

A closed case: everything readable, nothing changeable — by rule



Case #6 as dana: the evidence renders (welcome_email.png), and both forms are replaced by the sealing rule's name — “posting is refused right now by sealed_thread”, “new evidence is refused right now by sealed_thread”. THREAD (0) is itself correct: the only comment is a lead's internal note, invisible to customers even in the record. S16/S25 prove the seal universally; P13 proves the record stays readable. The seed had to write this thread BEFORE closing the case — sealed_thread refuses late seed data exactly as it refuses late users.

The internal flag, the robot's redact, the late post: 403 by name

```
# tests/test_relay.py — forge what the UI never renders
# dana is shown no internal checkbox — send the field:
fetch("POST", f"/case/{id}/comment", "dana",
      body="body=sneaky&internal=yes")
assert 403 and 'internal_is_staff_only'

# the robot forges a redact it was never shown:
fetch("POST", f"/comment/{cid}/act", "postbot",
      body="action=redact")
assert 403 and 'default_deny'

# posting into the seed's CLOSED case:
fetch("POST", f"/case/{closed}/attach", "dana",
      body="filename=late.log")
assert 403 and 'sealed_thread'
```

Three layers, one vocabulary

- Model: exhaustive two-backend agreement now spans all three entities — 37,200 situations, every rule, both the runtime evaluator and the Z3 compilation.
- Kernel: internal notes absent from customers' lists AND refused by forged id; org walls via the parent; live sealing on close; author-or-lead removal; robot contained on children.
- HTTP: the forged requests left — all 403, all naming the rule, nothing written. 19 tests; check.sh runs all of it plus the gate and the boundary lint in both directions.

This suite is why the UI's freedom costs nothing: what the client never renders, the kernel still refuses — under the same rule id the ticket, the gate, the solver counterexample and the toast all use.

Kept honest — each one is a falsifier with a design attached

Aggregates are not expressible (ME-1)

"Close only if no open comments" needs a parent rule over child sets. An aggregate atom is NOT a free boolean — it couples the parent's space to the child rules (a frame problem), so this is genuinely hard, not missing syntax. Sound cut when a ticket forces it: kernel-computed aggregate projections, runtime-sound, analyzer-conservative.

Parent-delete cascades skip child rules (ME-6)

Deleting a parent removes children WITHOUT consulting their delete rules — a child's immortality is only as strong as its parent's, and the per-entity gate cannot see that path. Pinned by an engine test; unreachable in Relay (nothing deletes). Fix candidates: cascade-decides, or a load-time compatibility check.

Field-level visibility (ME-3)

A redacted comment stays a readable row; the tombstone is the UI's choice — a rude free UI could render the body. Rules cannot say "readable except this field." Moving that inside the boundary costs per-field read atoms.

Depth and polymorphism (ME-2)

Attachments on comments (two levels), reactions on either (two parents). The context mechanism composes on paper; the space budget and the kernel's join path are where it may crack.

The join under load (ME-4)

Child visible() loads parent rows — cached per call, still N+1-shaped. Folds into KB2: a reporting seam that applies the read rule per row at SQL speed.

The fail-open lint (ME-5)

An actor-only deny not tagged for every entity deserves a load-time warning — slide 14's bug, caught before it ships. Cheap, mechanical, unbuilt.

Five rule bases, one engine, every claim executable

5 : 1

services per engine — CMS, tickets, receivables, Flowdeck, Relay on ~2,300 lines of domain-free Python (rule base 456, kernel 291, analyzer 351, lint 105).

3 entities

in one rule base — case, comment, attachment. Engine domain vocabulary added for the thread + evidence: 0 lines, the second zero in a row.

107,856

situations checked exhaustively across the five services — runtime evaluation and the Z3 compilation agree on every one (Relay alone: 37,200 over three entities).

2-way CI

check.sh holds both directions: Relay's gate must PASS, the preserved bypass variant must FAIL the lint, the taskboard's round-2 draft must FAIL the frozen gate.

Relay by the numbers — the whole product:

- 721 lines of YAML (33 rules + 29 $\forall$ -properties + 16 $\exists$ -witnesses + 4 gated lifecycle entries + 75 frozen feature steps across 7 features) — reviewed by the solver in ~0.4 s per change.
- 764 lines of free htmx UI (app.py) — reviewed by nobody, for policy, ever: the boundary lint proves it cannot touch state around the kernel.
- 19 tests across three layers; 5-stage check.sh; 8 reproducible screenshots (screenshots.py); journal with pre-registered predictions in DEVLOG.md.

What the analyzer asks of every rule change, in seconds, per entity: dead rules · stale assumptions · $\forall$ -safety · $\exists$ -possibility · lifecycle liveness + gated entries · frozen features with refusals by name.

Learned the hard way — the newest one is this deck's thesis

11 · Relations are engine work, never client joins

NEW. Child entities import parent context (parent.state, parent projections); the kernel joins the live parent into every child decision. Sharp edges kept: untagged global denies fail OPEN (tag + pin them); cascades skip child delete rules.

1 · The spec is frozen for agents

The analyzer takes --gate from a pinned directory; agents edit rules, never the gate. Historic buggy drafts are gated forever as regressions.

3 · Gate strength beats NL steering

Same bug, model, prompt: a stronger gate turned a partial fix into the full fix (P4). Cheaper models are fine when the gate is strong.

7 · Record falsified predictions

The DEVLOGs keep wrong predictions next to the runs that killed them, pre-registered predictions next to the runs that confirmed them — and slide 14's un-predicted lesson in between.

10 · The boundary is a kernel API, not a UI

Rules guard interaction logic behind a function-level API; the UI above is agent-authored and free; generated UIs are scaffolding. Hold the boundary mechanically (lint), never by convention.

2 · Gates need both directions

Safety-only gates accept fixes that trade away liveness (“the safest system does nothing”). Every gate = safety + possibility witnesses + frozen feature runs.

6 · Counterexamples are the currency

Named and machine-readable: situation witnesses, unsat cores, denied_by 403s — typed Denied values any UI renders its own way.

8 · Boundaries by construction

Where the language allows: capability tokens, typestate. Where it doesn't (Python): name-mangling + a lint whose failure is named — and the honesty that this is not a proof.

Falsifiers on the table

Held so far

- RB1 (tickets land as rule diffs): held five times, and the cost curve stayed closed — imports 0, receivables ~100 (time), Flowdeck ~60 (relations), Relay 0, Relay's thread + evidence 0 again.
- The one-entity wall came down as engine work with zero existing rules or gate lines weakened — and both multi-entity analyzer rounds were PASS with predictions pre-registered (plus one honest un-predicted lesson, slide 14).

Next

- ME-1 — aggregates, when a ticket forces them: kernel-computed aggregate projections, with the analyzer's conservatism stated. ME-6 — cascade-decides (parent delete consults child delete rules) or the load-time check.
- KB1 — red-team the boundary lint. KB2/ME-4 — a reporting/read seam (search, aggregation, pagination) that still applies the read rule per row. KB3 — concurrent writers between decide and write, on real Postgres (the P3 harness). KB4 — nearest-allow refusal explanations that do not leak across tenants.
- DX1 — an outside developer ships a ruled app (rules + free htmx UI) from tickets in under a day. DX2 — the same tickets implemented conventionally by a strong LLM: does it contain the interaction holes the gates caught?
- RB3 — ticket→rule-diff vs ticket→handler fidelity, measured. P8 — the same rules as Cedar policies. P9 — the invariant→Postgres-constraint compiler nobody has built.

Parked (consciously, with reasons recorded)

Alloy (Z3 covers it) · Quint→Rust codegen (no tooling) · Kani until unbounded domains arrive · Verus (proxy-blocked; Dafny stands in).

The one-sentence takeaway

One rule base now rules the whole product — cases, comments, attachments — because a child's rules see its parent's live state, and the kernel joins that parent into every decision: “a closed case seals its thread” is law checked in all 37,200 situations, not a flag the UI remembers to sync.

Run it: `examples/helpdesk/check.sh` · `python examples/helpdesk/app.py` → `http://127.0.0.1:8810/`

Read it: `examples/helpdesk/DEVLOG.md` · `research/16-multi-entity-rules.md` (and 15, the boundary beneath)

The stack beneath: notes 13–14 (rules as the program, the DX study) · 09–12 (proof escalation) · p1–p7